

ch01-intro

May 18, 2026

0.0.1 Chapter 1: Computing with Python

0.0.2 Overview: a typical Python-based scientific computing stack.



0.0.3 Resources:

- [Intel MKL \(Math Kernel Library\)](#)
- [openBLAS](#)
- [ATLAS](#)
- [SciPy](#)
- [Python Numeric & Scientific topics](#)

0.0.4 Interpreter

- The easiest way to execute Python code: run the program directly.
- Use Jupyter magic command to write Python source file to disk:

```
[1]: %%writefile hello.py
print("Hello from Python!")
```

Overwriting hello.py

- Use the ! system shell command (included in the Python Jupyter kernel) to interactively run Python with hello.py as its argument.

```
[2]: !python3 hello.py
```

Hello from Python!

```
[3]: !python3 --version
```

Python 3.12.3

0.0.5 Input and output caching

- Input & output history can be accessed using **In** (a list) & **Out** (a dictionary). Both can be indexed with a cell number.

```
[4]: 3*3
In[1]
```

```
[4]: 'get_ipython().run_cell_magic(\'writefile\', \'hello.py\', \'print("Hello from
Python!")\\n\\n\')
```

- A single underscore = the most recent output;
- A double underscore = the *next* most recent output.

```
[5]: 1+1
```

```
[5]: 2
```

```
[6]: 2+2
```

```
[6]: 4
```

```
[7]: _, _
```

```
[7]: (4, 2)
```

```
[8]: # In = a list
In
```

```
[8]: ['',
'get_ipython().run_cell_magic(\'writefile\', \'hello.py\', \'print("Hello from
Python!")\\n\\n\')',
'get_ipython().system(\'python3 hello.py\')',
'get_ipython().system(\'python3 --version\')',
'3*3\\nIn[1]',
```

```
'1+1',
'2+2',
'_', '--',
'# In = a list\nIn']
```

```
[9]: # Out = a dictionary
Out
```

```
[9]: {4: 'get_ipython().run_cell_magic(\'writefile\', \'hello.py\', \'print("Hello
from Python!")\\n\\')',
5: 2,
6: 4,
7: (4, 2),
8: [' ',
'get_ipython().run_cell_magic(\'writefile\', \'hello.py\', \'print("Hello from
Python!")\\n\\')',
'get_ipython().system(\'python3 hello.py\')',
'get_ipython().system(\'python3 --version\')',
'3*3\nIn[1]',
'1+1',
'2+2',
'_', '--',
'# In = a list\nIn',
'# Out = a dictionary\nOut']}
```

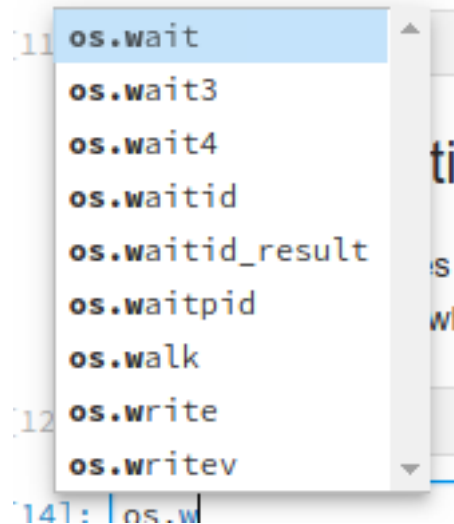
```
[10]: # Suppress output results by ending statement with a semicolon
1+2;
```

0.0.6 Autocompletion

- The **Tab** key activates autocompletion (displays list of symbol names that are valid completions of what has been typed thus far.)

```
[11]: import os
```

- Results of typing “os.w”, followed by ↵



0.0.7 Documentation

- “Docstrings” provide a built-in reference manual for most Python modules. Display the docstring by appending a Python object with “?”.

```
[12]: import math
      math.cos?
```

Signature: `math.cos(x, /)`

Docstring: Return the cosine of x (measured in radians).

Type: builtin_function_or_method

0.0.8 Interaction with System Shell

- Anything after ! is evaluated using the system shell, such as bash.
- (I use Ubuntu Linux as my laptop OS. Your Windows equivalents will vary.)

```
[13]: !touch file1.py file2.py file3.py
      !ls file*
```

file1.py file2.py file3.py file-io-NP.pdf

```
[14]: # output of a system shell command can be captured in a Python variable
      files = !ls file*
      print(len(files))
      print(files)
```

4

```
['file1.py', 'file2.py', 'file3.py', 'file-io-NP.pdf']
```

```
[15]: # pass Python variable values to shell commands
      # by prefixing the variable name with $.
      file = "file1.py"
      !ls -l $file
```

```
-rw-rw-r-- 1 bjpcjp www-data 0 Sep 27 18:11 file1.py
```

0.0.9 IPython Extensions

- Commands start with one or two “%” characters. A single % is used for single-line commands; dual %% is used for cells (multiple lines).
- %lsmagic returns a list of available commands.

```
[16]: %lsmagic
```

[16]: Available line magics:

```
%alias %alias_magic %autoawait %autocall %automagic %autosave %bookmark
%cat %cd %clear %code_wrap %colors %conda %config %connect_info %cp
%debug %dhist %dirs %doctest_mode %ed %edit %env %gui %hist %history
%killbgscripts %ldir %less %lf %lk %ll %load %load_ext %loadpy %logoff
%logon %logstart %logstate %logstop %ls %lsmagic %lx %macro %magic
%mamba %man %matplotlib %micromamba %mkdir %more %mv %notebook %page
%pastebin %pdb %pdef %pdoc %pfile %pinfo %pinfo2 %pip %popd %pprint
%precision %prun %psearch %psource %pushd %pwd %pycat %pylab %qtconsole
%quickref %recall %rehashx %reload_ext %rep %rerun %reset
%reset_selective %rm %rmdir %run %save %sc %set_env %store %sx %system
%tb %time %timeit %unalias %unload_ext %who %who_ls %whos %xdel %xmode
```

Available cell magics:

```
%%! %%HTML %%SVG %%bash %%capture %%code_wrap %%debug %%file %%html
%%javascript %%js %%latex %%markdown %%perl %%prun %%pypy %%python
%%python2 %%python3 %%ruby %%script %%sh %%svg %%sx %%system %%time
%%timeit %%writefile
```

Automagic is ON, % prefix IS NOT needed for line magics.

0.0.10 Running scripts

- %run executes an external Python source file within an interactive IPython session.

```
[17]: %%writefile fib.py
```

```
def fib(N):
    """
    Return a list of the first N Fibonacci numbers.
    """
    f0, f1 = 0, 1
    f = [1] * N
    for n in range(1, N):
        f[n] = f0 + f1
        f0, f1 = f1, f[n]

    return f
```

```
print(fib(10))
```

Overwriting fib.py

```
[18]: !python3 fib.py
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

```
[19]: %run fib.py
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

```
[20]: fib(6)
```

```
[20]: [1, 1, 2, 3, 5, 8]
```

0.1 Listing all defined symbols

- %who lists all defined symbols
- %whos provides more detailed info.

```
[21]: %who
```

```
fib      file      files      math      os
```

```
[22]: %whos
```

Variable	Type	Data/Info
fib	function	<function fib at 0x7bc3a66385e0>
file	str	file1.py
files	SList	['file1.py', 'file2.py', <...>e3.py', 'file-io-NP.pdf']
math	module	<module 'math' (built-in)>
os	module	<module 'os' (frozen)>

0.2 Debugger

- Use %debug to step directly into the Python debugger.

```
[23]: # fib function fails - can't use floating point numbers.  
try:  
    fib(1.0)  
except TypeError:  
    print("nope. can't do that.")
```

```
nope. can't do that.
```

```
[24]: #%debug
```

0.2.1 Resetting the Python namespace

- Ensures a program is run in a pristine environment, uncluttered by existing variables and functions. Although it is necessary to reimport modules, it is important to know that even if the modules have changed since the last import, a new import after a %reset will not import the new module but rather reenables a cached version from the previous import.
- When developing Python modules, this is usually not the desired behavior. In that case, a reimport of a previously imported (and since updated) module can often be achieved by using the reload function from `IPython.lib.deepreload`. However, this method does not always work, as some libraries run code at import time that is only intended to run once. In this case, the only option might be to terminate and restart the IPython interpreter.

```
[29]: %reset
```

Once deleted, variables cannot be recovered. Proceed (y/[n])?

Nothing done.

0.3 Timing and profiling code

- %timeit and %time provide simple benchmarking utilities.

```
[30]: # first, re-define fibonacci code used above.
```

```
def fib(N):  
    """  
    Return a list of the first N Fibonacci numbers.  
    """  
    f0, f1 = 0, 1  
    f = [1] * N  
    for n in range(1, N):  
        f[n] = f0 + f1  
        f0, f1 = f1, f[n]  
  
    return f
```

```
[31]: # timeit does not return expression's resulting value.
```

```
%timeit fib(50)
```

1.18 s ± 47.2 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)

```
[32]: # %time only runs once. less accurate estimate.
```

```
result = %time fib(100)
```

CPU times: user 7 s, sys: 0 ns, total: 7 s

Wall time: 9.3 s

```
[33]: len(result)
```

```
[33]: 100
```

- The **cProfile** module provides **%prun** (for statements) and **%run** (for external scripts) profiling commands.

```
[34]: import numpy as np

def random_walker_max_distance(M, N):
    """
    Simulate N random walkers taking M steps
    Return the largest distance from the starting point.
    """
    trajectories = [np.random.randn(M).cumsum()
                    for _ in range(N)]
    return np.max(np.abs(trajectories))
```

```
[35]: # returns call counts, runtime & cum runtime for each function.
%prun random_walker_max_distance(400, 10000)
```

20737 function calls (20731 primitive calls) in 0.251 seconds

Ordered by: internal time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
10000	0.153	0.000	0.153	0.000	{method 'randn' of 'numpy.random. mtrand.RandomState' objects}
10000	0.065	0.000	0.065	0.000	{method 'cumsum' of 'numpy. ndarray' objects}
1	0.020	0.020	0.229	0.229	220825864.py:
3					(random_walker_max_distance)
1	0.002	0.002	0.002	0.002	{method 'reduce' of 'numpy.ufunc' objects}
14	0.001	0.000	0.001	0.000	socket.py:626(send)
1	0.001	0.001	0.230	0.230	<string>:1(<module>)
1	0.000	0.000	0.011	0.011	history.py:
833					(_writeout_input_cache)
2	0.000	0.000	0.000	0.000	{method 'recv' of '_socket.socket' objects}
4	0.000	0.000	0.249	0.062	selectors.py:451(select)
2	0.000	0.000	0.004	0.002	zmqstream.py:584(_handle_events)
2	0.000	0.000	0.003	0.002	socket.py:703(send_multipart)
16	0.000	0.000	0.001	0.000	enum.py:1551(__or__)
72	0.000	0.000	0.000	0.000	enum.py:1544(_get_value)
2	0.000	0.000	0.003	0.002	iostream.py:278(_really_send)
152/148	0.000	0.000	0.000	0.000	{built-in method builtins. isinstance}
5	0.000	0.000	0.000	0.000	attrsettr.py:66(_get_attr_opt)
31	0.000	0.000	0.000	0.000	enum.py:726(__call__)

5	0.000	0.000	0.001	0.000	attrsettr.py:43(__getattr__)
8	0.000	0.000	0.000	0.000	enum.py:1562(__and__)
2	0.000	0.000	0.000	0.000	socket.py:778(recv_multipart)
5	0.000	0.000	0.007	0.001	{method 'run' of '_contextvars'.
↳Context' objects}					
31	0.000	0.000	0.000	0.000	enum.py:1129(__new__)
4	0.000	0.000	0.255	0.064	base_events.py:1910(_run_once)
1	0.000	0.000	0.000	0.000	inspect.py:3102(_bind)
1	0.000	0.000	0.011	0.011	history.py:845(writeout_cache)
1	0.000	0.000	0.000	0.000	selector_events.py:
↳129(_read_from_self)					
4	0.000	0.000	0.000	0.000	typing.py:1258(__hash__)
3	0.000	0.000	0.000	0.000	events.py:36(__init__)
6/5	0.000	0.000	0.007	0.001	events.py:86(_run)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
2	0.000	0.000	0.000	0.000	<frozen abc>:121(__subclasscheck__)
3	0.000	0.000	0.000	0.000	threading.py:299(__enter__)
2	0.000	0.000	0.000	0.000	base_events.py:785(call_soon)
3	0.000	0.000	0.001	0.000	zmqstream.py:687(_update_handler)
1	0.000	0.000	0.000	0.000	traitlets.py:
↳1527(_notify_observers)					
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
6	0.000	0.000	0.000	0.000	typing.py:392(inner)
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof'.
↳Profiler' objects}					
1	0.000	0.000	0.000	0.000	kernelbase.py:
↳302(poll_control_queue)					
1	0.000	0.000	0.000	0.000	{method 'execute' of 'sqlite3'.
↳Connection' objects}					
2	0.000	0.000	0.000	0.000	base_events.py:1895(_add_callback)
1	0.000	0.000	0.000	0.000	poll.py:80(poll)
8	0.000	0.000	0.000	0.000	traitlets.py:676(__get__)
1	0.000	0.000	0.000	0.000	queues.py:317(_put_internal)
3	0.000	0.000	0.001	0.000	zmqstream.py:664(_rebuild_io_state)
4	0.000	0.000	0.000	0.000	queue.py:97(empty)
2	0.000	0.000	0.002	0.001	zmqstream.py:557(_run_callback)
1	0.000	0.000	0.018	0.018	history.py:55(only_when_enabled)
2	0.000	0.000	0.003	0.002	iostream.py:276(<lambda>)
2/1	0.000	0.000	0.000	0.000	{method 'poll' of 'select.epoll'}
↳objects}					
1	0.000	0.000	0.001	0.001	kernelbase.py:324(_flush)
5	0.000	0.000	0.000	0.000	<frozen importlib._bootstrap>:
↳1390(_handle_fromlist)					
3	0.000	0.000	0.000	0.000	base_events.py:814(_call_soon)
8	0.000	0.000	0.000	0.000	traitlets.py:629(get)
1	0.000	0.000	0.000	0.000	decorator.py:199(fix)
6	0.000	0.000	0.000	0.000	{built-in method builtins.next}

1	0.000	0.000	0.000	0.000	asyncio.py:225(add_callback)
2	0.000	0.000	0.003	0.001	zmqstream.py:625(_handle_recv)
2	0.000	0.000	0.000	0.000	traitlets.py:689(set)
1	0.000	0.000	0.000	0.000	zmqstream.py:427(flush)
1	0.000	0.000	0.000	0.000	_base.py:537(set_result)
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
1	0.000	0.000	0.000	0.000	queues.py:225(get)
10	0.000	0.000	0.000	0.000	{built-in method builtins.hasattr}
1	0.000	0.000	0.000	0.000	inspect.py:3237(bind)
1	0.000	0.000	0.000	0.000	{method 'send' of '_socket.socket'}
↳objects}					
1	0.000	0.000	0.000	0.000	{built-in method _asyncio.
↳get_running_loop}					
22	0.000	0.000	0.000	0.000	{built-in method builtins.len}
2	0.000	0.000	0.000	0.000	typing.py:1465(__subclasscheck__)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
1	0.000	0.000	0.000	0.000	threading.py:311(_acquire_restore)
1	0.000	0.000	0.248	0.248	{built-in method builtins.exec}
1	0.000	0.000	0.000	0.000	traitlets.py:1523(notify_change)
4	0.000	0.000	0.000	0.000	selector_events.py:
↳750(_process_events)					
5	0.000	0.000	0.000	0.000	{built-in method builtins.getattr}
1	0.000	0.000	0.000	0.000	inspect.py:2918(apply_defaults)
1	0.000	0.000	0.000	0.000	queues.py:209(put_nowait)
5	0.000	0.000	0.000	0.000	base_events.py:734(time)
2	0.000	0.000	0.000	0.000	{method '__exit__' of 'sqlite3.
↳Connection' objects}					
2	0.000	0.000	0.000	0.000	{built-in method posix.getpid}
5	0.000	0.000	0.000	0.000	{built-in method builtins.max}
2	0.000	0.000	0.000	0.000	typing.py:1194(__instancecheck__)
2	0.000	0.000	0.002	0.001	iostream.py:157(_handle_event)
2	0.000	0.000	0.000	0.000	traitlets.py:
↳3624(validate_elements)					
1	0.000	0.000	0.018	0.018	decorator.py:229(fun)
1	0.000	0.000	0.000	0.000	queues.py:186(put)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
1	0.000	0.000	0.000	0.000	zmqstream.py:468(update_flag)
1	0.000	0.000	0.000	0.000	inspect.py:2865(args)
2	0.000	0.000	0.000	0.000	iostream.py:213(_is_master_process)
2	0.000	0.000	0.001	0.001	ioloop.py:742(_run_callback)
9	0.000	0.000	0.000	0.000	{method 'popleft' of 'collections.
↳deque' objects}					
2	0.000	0.000	0.000	0.000	{built-in method builtins.
↳issubclass}					
1	0.000	0.000	0.004	0.004	asyncio.py:200(_handle_events)
28	0.000	0.000	0.000	0.000	typing.py:2154(cast)
2	0.000	0.000	0.000	0.000	traitlets.py:708(__set__)

3	0.000	0.000	0.000	0.000	threading.py:302(__exit__)
2	0.000	0.000	0.000	0.000	{built-in method _abc.
↪_abc_subclasscheck}					
1	0.000	0.000	0.000	0.000	futures.py:394(_call_set_state)
6	0.000	0.000	0.000	0.000	{method 'append' of 'list' objects}
8	0.000	0.000	0.000	0.000	{method 'append' of 'collections.
↪deque' objects}					
8	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.
↪lock' objects}					
4	0.000	0.000	0.000	0.000	zmqstream.py:543(sending)
1	0.000	0.000	0.000	0.000	threading.py:424(notify_all)
1	0.000	0.000	0.000	0.000	base_events.py:
↪838(call_soon_threadsafe)					
3	0.000	0.000	0.000	0.000	{method 'acquire' of '_thread.
↪lock' objects}					
1	0.000	0.000	0.002	0.002	fromnumeric.py:71(_wrapreduction)
1	0.000	0.000	0.000	0.000	concurrent.py:
↪181(future_set_result_unless_cancelled)					
2	0.000	0.000	0.000	0.000	traitlets.py:2304(validate)
5	0.000	0.000	0.000	0.000	{method 'upper' of 'str' objects}
2	0.000	0.000	0.000	0.000	queues.py:322(_consume_expired)
2	0.000	0.000	0.000	0.000	{method 'set_result' of '_asyncio.
↪Future' objects}					
5	0.000	0.000	0.000	0.000	{built-in method time.monotonic}
1	0.000	0.000	0.000	0.000	queues.py:256(get_nowait)
1	0.000	0.000	0.000	0.000	selector_events.py:
↪141(_write_to_self)					
2	0.000	0.000	0.000	0.000	selectors.py:275(_key_from_fd)
1	0.000	0.000	0.000	0.000	_base.py:337(_invoke_callbacks)
1	0.000	0.000	0.000	0.000	inspect.py:288(kwargs)
4	0.000	0.000	0.000	0.000	queue.py:209(_qsize)
1	0.000	0.000	0.000	0.000	threading.py:627(clear)
2	0.000	0.000	0.000	0.000	iostream.py:216(_check_mp_mode)
1	0.000	0.000	0.000	0.000	history.py:
↪839(_writeout_output_cache)					
4	0.000	0.000	0.000	0.000	{built-in method builtins.hash}
1	0.000	0.000	0.000	0.000	threading.py:308(_release_save)
1	0.000	0.000	0.000	0.000	threading.py:394(notify)
2	0.000	0.000	0.000	0.000	{built-in method _contextvars.
↪copy_context}					
3	0.000	0.000	0.000	0.000	{method 'items' of 'mappingproxy' ↵
↪objects}					
4	0.000	0.000	0.000	0.000	{method 'get' of 'dict' objects}
1	0.000	0.000	0.002	0.002	fromnumeric.py:2692(max)
5	0.000	0.000	0.000	0.000	zmqstream.py:539(receiving)
2	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.
↪lock' objects}					

2	0.000	0.000	0.000	0.000	traitlets.py:
↳3486(validate_elements)					
10	0.000	0.000	0.000	0.000	inspect.py:2777(kind)
1	0.000	0.000	0.000	0.000	poll.py:31(register)
1	0.000	0.000	0.000	0.000	{built-in method builtins.min}
1	0.000	0.000	0.001	0.001	zmqstream.py:695(<lambda>)
1	0.000	0.000	0.000	0.000	{method '__enter__' of '_thread.
↳RLock' objects}					
1	0.000	0.000	0.000	0.000	queues.py:173(qsize)
1	0.000	0.000	0.000	0.000	{method 'values' of 'mappingproxy'}
↳objects}					
5	0.000	0.000	0.000	0.000	base_events.py:2005(get_debug)
2	0.000	0.000	0.000	0.000	iostream.py:255(closed)
1	0.000	0.000	0.000	0.000	{built-in method math.ceil}
1	0.000	0.000	0.000	0.000	unix_events.py:
↳81(_process_self_data)					
3	0.000	0.000	0.000	0.000	base_events.py:539(_check_closed)
1	0.000	0.000	0.000	0.000	queues.py:312(_put)
1	0.000	0.000	0.000	0.000	queues.py:309(_get)
2	0.000	0.000	0.000	0.000	{built-in method builtins.iter}
4	0.000	0.000	0.000	0.000	inspect.py:3058(parameters)
2	0.000	0.000	0.000	0.000	{method 'extend' of 'list' objects}
2	0.000	0.000	0.000	0.000	{method 'cancelled' of '_asyncio.
↳Future' objects}					
1	0.000	0.000	0.000	0.000	{built-in method _thread.
↳allocate_lock}					
4	0.000	0.000	0.000	0.000	inspect.py:2765(name)
1	0.000	0.000	0.000	0.000	threading.py:314(_is_owned)
1	0.000	0.000	0.000	0.000	queues.py:177(empty)
1	0.000	0.000	0.000	0.000	{method 'done' of '_asyncio.
↳Future' objects}					
1	0.000	0.000	0.000	0.000	zmqstream.py:660(_check_closed)
1	0.000	0.000	0.000	0.000	{method 'release' of '_thread.
↳lock' objects}					
1	0.000	0.000	0.000	0.000	locks.py:224(clear)
1	0.000	0.000	0.000	0.000	{method '_is_owned' of '_thread.
↳RLock' objects}					
1	0.000	0.000	0.000	0.000	{method '__exit__' of '_thread.
↳RLock' objects}					
1	0.000	0.000	0.000	0.000	inspect.py:2857(__init__)
1	0.000	0.000	0.000	0.000	base_events.py:720(is_closed)
1	0.000	0.000	0.000	0.000	queues.py:59(_set_timeout)
1	0.000	0.000	0.000	0.000	fromnumeric.py:
↳2687(_max_dispatcher)					
1	0.000	0.000	0.000	0.000	{method 'items' of 'dict' objects}

0.3.1 Jupyter: External image rendering

```
[36]: from IPython.display import display, Image, HTML, Math
```

```
[37]: Image(url='http://python.org/images/python-logo.gif')
```

```
[37]: <IPython.core.display.Image object>
```

0.3.2 Jupyter: HTML rendering

```
[35]: import scipy, numpy, matplotlib
modules = [numpy, matplotlib, scipy]

row = "<tr><td>%s</td><td>%s</td></tr>"
rows = "\n".join(
    [row %
     (module.__name__, module.__version__)
     for module in modules])
table = "<table><tr><th>Library</th><th>Version</th></tr> %s </table>" % rows
```

```
[36]: HTML(table)
```

```
[36]: <IPython.core.display.HTML object>
```

```
[37]: # another method
class HTMLdisplayer(object):
    def __init__(self, code):
        self.code = code
    def _repr_html_(self):
        return self.code

HTMLdisplayer(table)
```

```
[37]: <__main__.HTMLdisplayer at 0x7f3940100fd0>
```

0.3.3 Jupyter: Formula rendering using Latex

```
[38]: Math(r'\hat{H} = -\frac{1}{2}\epsilon\sigma_z - \frac{1}{2}\delta\sigma_x \rightarrow \hat{\sigma}_x')
```

```
[38]: 
$$\hat{H} = -\frac{1}{2}\epsilon\hat{\sigma}_z - \frac{1}{2}\delta\hat{\sigma}_x$$

```

0.3.4 Jupyter: UI Widgets

```
[39]: import matplotlib.pyplot as plt
import numpy as np
from scipy import stats
```

```
def f(mu):
    X = stats.norm(loc=mu, scale=np.sqrt(mu))
    N = stats.poisson(mu)
    x = np.linspace(0, X.ppf(0.999))
    n = np.arange(0, x[-1])

    fig, ax = plt.subplots()
    ax.plot(x, X.pdf(x), color='black', lw=2, label="Normal( $\mu$ =%d,  $\sigma^2$ =%d)" % (mu, mu))
    ax.bar(n, N.pmf(n), align='edge', label=r"Poisson( $\lambda$ =%d)" % mu)
    ax.set_ylim(0, X.pdf(x).max() * 1.25)
    ax.legend(loc=2, ncol=2)
    plt.close(fig)
    return fig
```

```
[40]: from ipywidgets import interact
import ipywidgets as widgets
```

```
[42]: interact(f, mu=widgets.FloatSlider(min=1.0, max=20.0, step=1.0));
```

```
interactive(children=(FloatSlider(value=1.0, description='mu', max=20.0, min=1.0, step=1.0), Output()), _dom_c...
```

0.3.5 nbconvert to HTML

```
[43]: !jupyter nbconvert --to html ch01-intro.ipynb
```

```
[NbConvertApp] Converting notebook ch01-intro.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 1 image(s).
[NbConvertApp] Writing 348064 bytes to ch01-intro.html
```

0.3.6 nbconvert to PDF

- Requires a [LaTeX environment](#) to be installed.
- On this system (Ubuntu Linux): `sudo apt-get install texlive-xetex`

```
[45]: # be sure to install pandoc first.
!jupyter nbconvert --to pdf ch01-intro.ipynb;
```

```
[NbConvertApp] Converting notebook ch01-intro.ipynb to pdf
[NbConvertApp] Writing 63377 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 128251 bytes to ch01-intro.pdf
```

```
[46]: !ls ch01*
```

```
ch01-intro.html  ch01-intro.ipynb  ch01-intro.pdf  ch01-intro.py
```